# Prime Map API 1.0 — Referans

Herhangi bir X-Plane 11 eklentisine gömülebilen harita kütüphanesi · 2026-11-10

## 1. Tanım

Prime Map API, X-Plane 11 eklentilerinin kendi arayüzlerine katmanlı bir hareketli harita yerleştirmesini sağlayan bir C++ kütüphanesidir (SHARED DLL: `prime_map.dll`). Harita, verisini sadece X-Plane'den alır: kurulu DSF sahneleri (zemin, su, yollar), apt.dat (havalimanları), nav veritabanı, `earth_awy.dat` (hava yolları), dataref'ler (uçak) ve FMS (rota). Harita kendi XPLM penceresinde, kendi flight loop'uyla çizilir; host eklenti pencereyi açar, konumlandırır ve bütün davranışı API fonksiyonlarıyla yönetir. Haritanın yerleşik arayüzü (araç şeridi, katman paneli, mod paneli, çizim paleti, havalimanı arama, durum şeridi, tooltip) isteğe bağlıdır.

| Özellik | Değer |
|---|---|
| Platform | X-Plane 11, Windows x64, SDK 3.0 (XPLM301). Host her zaman bir X-Plane eklentisidir. |
| Dil, derleyici | C++20, MSVC 2022 |
| Ad alanı / başlık | `prime` / tek başlık `PrimeMap.h` (`#include <prime_map_api/PrimeMap.h>`; sadece standart C++ tipleri) |
| Dağıtım biçimi | SHARED DLL (`prime_map.dll`, `/MD` CRT) + import kütüphanesi `prime_map.lib`; DLL host eklentinin yanına kopyalanır. Hedef makinede MSVC 2022 redistributable gerekir |
| Harita sayısı | Çoklu. Aynı anda birden çok açık `Map` penceresi olabilir; her harita kendi penceresiyle bağımsızdır. DLL ortak kütüphane olduğu için aynı X-Plane sürecindeki **tüm hostlar** aynı kütüphane nesnesini (`prime::MapLibrary::Get()`) paylaşır: `Init` bir kez yapılır, sonraki hostlar `true` alır (ilk hostun yapılandırması geçerlidir); kapanış referans sayaçlıdır |
| Thread | Bütün fonksiyonlar X-Plane ana thread'inden çağrılır |
| Hata modeli | İstisna yok. Geçersiz değer: işlem yapılmaz, Log'a bir satır uyarı, dönüş `false` / boş |
| Dosyalar | Kütüphane ayar/çizim dosyası yazmaz (dışa/içe aktarma metinleri verir); sadece önbellek klasörüne yazar |

## 2. Hızlı başlangıç

Var olan bir X-Plane 11 eklentisine harita eklemek beş adımdır. Fonksiyonların tam listesi bölüm 5'tedir.

### Adım 1 — Kütüphane reposunu edinin

`prime_map_api` reposunu eklenti projesinin yanına alın (kardeş klasör). İçeriği: genel başlık (`PrimeMap.h`, `primemapglobal.h`), CMake kütüphane tanımı, örnek eklenti (`examples/map_api_test`), bu belge. Sürüm notları `CHANGELOG.md`'dedir. SDK paketi üretilmez; tüketim CMake hedefi üzerinden yapılır (Adım 2).

### Adım 2 — Derlemeye bağlayın

Gereksinimler: C++20, MSVC 2022 (**/MD CRT** — kütüphane ile aynı olmak zorunda), X-Plane SDK 3.0. X-Plane SDK'sı developer.x-plane.com/sdk adresinden indirilir (geliştirme ortamına göre `C:\workspace\cots\X-Plane-SDK` altına kurulur; repoda tutulmaz). Kütüphane aynı CMake oturumuna katılır ve hedef adına bağlanır:

```
set(PRIME_MAP_DIR "${CMAKE_SOURCE_DIR}/../prime_map_api" CACHE PATH "prime_map_api klasoru")
if(NOT EXISTS "${PRIME_MAP_DIR}/PrimeMap.h")
    message(FATAL_ERROR "prime_map_api bulunamadi: ${PRIME_MAP_DIR} (kardes klasor olarak
klonlayin)")
endif()
add_subdirectory(${PRIME_MAP_DIR} prime_map_build)
target_link_libraries(host PRIVATE prime_map          # include yolu transitif gelir
                      ${XPSDK}/Libraries/Win/XPLM_64.lib opengl32)
target_compile_definitions(host PRIVATE
    XPLM200=1 XPLM210=1 XPLM300=1 XPLM301=1   # SDK 3.0 seviyesi
    IBM=1 APL=0 LIN=0                          # Windows; APL/LIN diger platformlar
```

Host kodunda başlık: `#include <prime_map_api/PrimeMap.h>`. Derleme sonrası `prime_map.dll`, host eklentinin (`win.xpl`) yanına kopyalanmalıdır (Windows DLL arama kuralı; X-Plane'de `Resources/plugins/<host>/64/`); kopyayı POST_BUILD ile otomatikleştirin:

```
add_custom_command(TARGET host POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_if_different
            $<TARGET_FILE:prime_map> $<TARGET_FILE_DIR:host>)
```

`/MD` nedeniyle hedef makinede MSVC 2022 redistributable (`VC_redist.x64`) kurulu olmalıdır.

### Adım 3 — Yaşam döngüsünü bağlayın

Kütüphane `XPluginEnable`'da başlatılır ve harita açılır; `XPluginDisable`'da kapatılır. En küçük çalışan iskelet:

```
#include <prime_map_api/PrimeMap.h>

static std::unique_ptr<prime::Map> gMap;

PLUGIN_API int XPluginEnable(void)
{
    prime::InitConfig tInit;
    // mCacheDirectory bos birakilirse: <eklenti klasoru>/cache/
    if (!prime::MapLibrary::Get().Init(tInit))
        return 0;

    prime::OpenConfig tOpen;
    tOpen.mGeometry = { 100, 800, 640, 480 };   // sol, ust, genislik, yukseklik (boxel)
    tOpen.mUiVisible = true;                    // yerlesik arayuz (araç seridi, katman paneli ...)
    tOpen.mMouseInput = true;                   // fare ile kaydirma ve zoom
    gMap = prime::MapLibrary::Get().CreateMap(tOpen);   // veya: Map::Open(tOpen)
    return 1;
}

PLUGIN_API void XPluginDisable(void)
{
    gMap.reset();            // pencere kapanir
    prime::MapLibrary::Get().Shutdown();    // arka plan isleri durur
}
```

Veri yüklemesi arka planda sürer; harita hemen açılabilir, katmanlar veri geldikçe tamamlanır (bkz. bölüm 4). Kütüphane ayar/çizim dosyası **yazmaz**: pencere yeri, mod, katman görünürlükleri ve kullanıcı çizimleri bellekte tutulur; kaydetmek host'un görevidir. Kütüphane bunun için `ExportSettings()`/`ImportSettings()`, `ExportDrawings()`/`ImportDrawings()` metin fonksiyonlarını ve her değişiklikte artan

`GetSettingsRevision()`/`GetDrawingsRevision()` sayaçlarını verir (bölüm 6, "Ayarlar" ve "Kullanıcı çizimleri"); host uygun anda dosyaya yazar ve açılışta geri yükler.

### Adım 4 — Pencereyi host arayüzüne yerleştirin

Embedded stilde harita çerçevesiz çizilir; konumunu ve boyutunu host belirler. Host penceresi her taşındığında/boyutlandığında `map->SetGeometry(...)` çağrılır (koordinat kuralı: bölüm 4). Düğme/klavye bağlamak isteyen host `PanLeft/Right/Up/Down`, `ZoomIn/ZoomOut`, `SetMode`, `SetLayerVisible` gibi fonksiyonları kendi kontrollerine bağlar.

### Adım 5 — Doğrulayın

Harita penceresi görünür ve durum şeridi veri yüklemesini gösterir. Başarısızlıkta işlemler sessizce yapılmaz; neden Log.txt'ye `mLogPrefix` ile yazılır (`InitConfig::mLogCallback` ile log'ları kendi dosyanıza da alabilirsiniz). Yükleme durumu için `prime::MapLibrary::Get().GetStatus()`, sürüm uyumu için `prime::VersionString()` kullanın. Yeni bir SDK sürümünde kütüphane reposunu güncellemeniz yeterlidir; API uyumsuzluğu major sürüm artışıyla bildirilir.

### Sık karşılaşılan durumlar

| Belirti | Sebep / çözüm |
|---|---|
| `Map::Open` / `CreateMap` nullptr döner | `MapLibrary::Get().Init(...)` çağrılmadı; önce `IsInitialized()` kontrol edin |
| `Init` false döner | Önbellek klasörü yazılabilir değil (`mCacheDirectory` boşsa eklenti klasörü altındaki `cache/`). Klasörü kontrol edin ya da yazılabilir bir yol verin |
| Pencere hiç açılmıyor | Pencere gizli olabilir (`OpenConfig::mVisible`, `SetVisible`); geometri ekran dışında olabilir (bölüm 5: left/top/right/bottom kuralı) |
| Zemin gri / boş kareler | Karoya kurulu sahne yok: o karo deniz veya "veri yok"tur; alt şeritte `Loading terrain` yazıyorsa yükleme devam ediyor. Kurulu sahneler simülatördekiyle aynıdır, kütüphane ayrı veri indirmez |
| Katmanlar / mod ayarları her açılışta sıfırlanıyor | Host kaydetmiyor: `ExportSettings` + revizyon sayaçları (bölüm 6). Kütüphane dosya yazmaz |
| Neden log dosyası bulamıyorum | Kütüphane dosyaya yazmaz; satırlar X-Plane'in `Log.txt`'sine `mLogPrefix` ("[prime-map]") ile yazılır. Kendi dosyanıza almak için `InitConfig::mLogCallback` |
| Fonksiyon sessizce iş yapmıyor | Tasarlanan hata modeli: geçersiz değer işleme alınmaz, Log'a uyarı yazılır, dönüş false/boş. Detaylı görmek için `SetDiagnostics(true)` |
| Link hatası: tanımlanmamış XPLM sembolleri | `XPLM_64.lib` bağlanmamış |
| Host açılışta `prime_map.dll` bulunamıyor | DLL host eklentinin (`win.xpl`) yanında değil; aynı `64/` klasörüne kopyalayın. Ya da host CRT'si kütüphaneden farklı: iki taraf da `/MD` olmalı |

## 3. Kütüphane ve bağlama (ekip kalıbı)

| Dosya | İçerik |
|---|---|
| `PrimeMap.h` | Genel başlık (repo kökünde; `#include <prime_map_api/PrimeMap.h>`) |
| `primemapglobal.h` | Export makrosu (`PRIME_MAP_EXPORT`; tüketici tarafında dllimport olur) |
| `prime_map.dll` + `prime_map.lib` | SHARED kütüphane ve import kütüphanesi (link otomatiktir: hedef adına bağlanır) |
| `examples/map_api_test/` | Bütün fonksiyonları çağıran örnek/test eklentisi (ayrı CMake projesi) |
| `docs/PrimeMap_API.pdf`, `CHANGELOG.md` | Bu belge, sürüm notları |

Host, kütüphane reposunu `add_subdirectory` ile kendi oturumuna katar, `prime_map` hedefine bağlanır ve `XPLM_64.lib + opengl32`'yi bağlar. LZMA kütüphanenin içindedir; yazı tipi X-Plane'in `Resources/fonts/DejaVuSans.ttf` dosyasından okunur. Kütüphane `/MD` CRT ile derlenir; host da `/MD` ile derlenmelidir (STL tipleri DLL sınırından geçer).

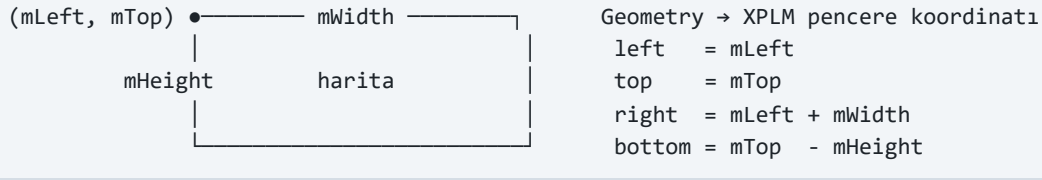
Kütüphanenin derlenmesi: `cmake -B b -DCMAKE_BUILD_TYPE=Release`, `cmake --build b --target prime_map` (sonuç: `prime_map.dll`). Paket/install adımı yoktur; tüketim CMake hedefi üzerinden yapılır.

## 4. Yaşam döngüsü

```
XPluginEnable :  auto& lib = prime::MapLibrary::Get();     kutuphane nesnesi
                 lib.Init(config)                          veri yüklemesi başlar (arka planda)
                 auto map = lib.CreateMap(openConfig)      harita penceresi açılır (aynı anda birden
çok harita olabilir)
                 (Map::Open aynı işi yapar; geriye uyumluluk için durur)
çalışırken    :  map->SetGeometry(...), map->ZoomIn(), map->SetMode(...) ...
XPluginDisable:  map.reset()                               pencere kapanır
                 lib.Shutdown()                            arka plan işleri durur (referans
sayacılı)
```

Veri yüklenirken harita hemen açılabilir; hazır olmayan katmanlar boş/kaba çizilir ve veri geldikçe tamamlanır (`GetStatus`). Zemin ayarı (`ImportTerrainSettings`/`SetTerrainSettings`), `CreateMap`'ten hemen sonra ve ilk kareden önce verilirse tam etkilidir; sonradan verilen zemin ayarı bir sonraki X-Plane açılışında etkili olur.

## 5. Koordinatlar ve birimler




| Büyüklük | Birim |
|---|---|
| Ekran konumu, boyut, `PanBy` | X-Plane global ekran birimi (boxel); `XPLMGetWindowGeometry` ile aynı. x sağa, y yukarı artar. UI ölçeği %100'de 1 boxel = 1 piksel, %150'de 1,5 piksel; dönüşümü kütüphane yapar. |
| Coğrafi konum | Derece (enlem kuzey +, boylam doğu +), WGS84 |
| Zoom | Web Mercator seviyesi, 3,0 … 16,5 (dünya kenarı 256 × $2^{zoom}$ boxel) |
| Yön, hız, irtifa | Gerçek yön (derece), knot, feet MSL |

## 6. Fonksiyonlar

| Fonksiyon | Ne yapar | Dönüş / kural |
|---|---|---|
| **Kütüphane (serbest fonksiyonlar)** | | |
| `Init(InitConfig)` | Kütüphaneyi başlatır, veri yüklemesini başlatır. `mCacheDirectory` (boş: *eklenti klasörü*/cache/), `mLogPrefix` ("[prime-map]"), `mLogCallback` (verilirse log satırları buraya), `mTerrainCacheMb` (1024; 0 sınırsız), `mPrefetchTiles` (20; 0 kapalı) | `bool`; ikinci çağrı false |
| `Shutdown()` | Arka plan işlerini durdurur; açık harita varsa kapatır | |

| | | |
|---|---|---|
| `IsInitialized()` | Kütüphane başlatıldı mı | `bool` |
| `GetStatus()` | Yükleme durumu: nav, havalimanı, hava yolu, sahne indeksi hazır mı; bekleyen zemin görüntüsü; ön önbellek ilerlemesi; durum metni; teşhis açıksa performans metni | `Status` |
| `VersionString()`, `kVersionMajor/Minor/Patch` | Kütüphane sürümü (CMake `project(VERSION)` ile aynıdır; configure'da denetlenir) | "0.22.0"; uyumsuz değişiklikte major artar |
| **Harita penceresi** | | |
| `Map::Open(OpenConfig)` | Haritayı açar. `mGeometry`, `mStyle` (Embedded), `mMode` (Scenery), `mUiVisible` (false), `mMouseInput` (false), `mVisible` (true), `mImageCache` (true), `mMaxRenderFps` (15), `mXPlaneFont` (false) | `unique_ptr<Map>`; Init yoksa veya harita açıksa nullptr |
| `~Map()` | Pencereyi kapatır | |
| `SetGeometry(Geometry)` / `GetGeometry()` | Pencereyi taşır ve boyutlandırır / konumunu verir | Her an çağrılabilir |
| `SetVisible(bool)` / `IsVisible()` | Pencereyi gösterir/gizler | |
| `BringToFront()` | Pencereyi diğer pencerelerin önüne alır | Host penceresi öne gelince çağrılır |
| `SetPoppedOut(bool)` / `IsPoppedOut()` | Pencereyi işletim sistemi penceresine çıkarır / geri alır | Sadece Standalone |
| `SetUiVisible(bool)` / `IsUiVisible()` | Yerleşik arayüzü açar/kapatır | |
| `SetMouseInput(bool)` / `IsMouseInputEnabled()` | Fare ile kaydırma, tekerlekle zoom, tooltip, çizim tıklaması | Kapalıyken tıklamalar alttaki pencereye geçer |
| **Görünüm** | | |
| `SetCenter(lat, lon)` / `GetCenter()` | Harita merkezini ayarlar / verir | Uçak takibini kapatır |
| `SetZoom(z)` / `GetZoom()` | Zoom seviyesi | `MinZoom()`..`MaxZoom()`'a kırpılır |
| `MinZoom()`, `MaxZoom()` | Zoom sınırları (statik) | 3,0 / 16,5 |
| `ZoomIn()`, `ZoomOut()` | Merkez etrafında bir adım yakınlaştırır/uzaklaştırır | |
| `SetZoomStep(step)` | ZoomIn/Out adımı | Varsayılan 0,5; (0, 5] |
| `PanLeft()`, `PanRight()`, `PanUp()`, `PanDown()` | Görünümü batı/doğu/kuzey/güneye kaydırır | Takibi kapatır |
| `SetPanStep(fraction)` | Pan adımı, görünen alanın oranı | Varsayılan 0,25; (0, 1] |
| `PanBy(dx, dy)` | Harita içeriğini boxel cinsinden kaydırır (dx > 0 sağa, dy > 0 yukarı) | Takibi kapatır |
| `FitBounds(GeoBounds)` | Alanın tamamını (kenarlarda %5 payla) gösteren merkez ve en büyük zoom'u ayarlar | Takibi kapatır |
| `GetVisibleBounds()` | Görünen alan | `GeoBounds` |
| `GetMetersPerBoxel()` | Merkez enlemindeki ölçek | metre/boxel |
| `SetFollowAircraft(bool)` / `IsFollowingAircraft()` | Haritayı her karede uçağa ortalar | Kaydırma takibi kapatır |
| `CenterOnAircraft()` | Bir kez uçağa ortalar | |
| `GoToAirport(ident)` | Kimliği tam eşleşen havalimanını ortalar ve sığdırır (uçak konumu değişmez) | `bool`; bulunamazsa false |

| | | |
|---|---|---|
| `FindAirports(query, max)` | Kimlik, ICAO kodu ve ada göre arama, en iyi eşleşme önce | `vector<AirportMatch>` (ident, ad, konum) |
| `ScreenToGeo(x, y)` | Ekran noktası → enlem/boylam | `LatLon` |
| `GeoToScreen(lat, lon, &x, &y)` | Enlem/boylam → ekran noktası | Pencere dışında false |
| **Harita modu** | | |
| `SetMode(MapMode)` / `GetMode()` | Scenery, Vfr, IfrLow, IfrHigh, Dark. Katman görünürlüğü mod başına saklanır | |
| `NextMode()` | Sıradaki moda geçer (döngüsel) | |
| **Katmanlar (kurallar bölüm 7)** | | |
| `SetLayerVisible(MapLayer, bool)` | Grup, katman veya alt katmanı açar/kapatır | |
| `IsLayerVisible(MapLayer)` | Öğe açık mı (alt katman: üstü de açık mı) | `bool` |
| `ToggleLayer(MapLayer)` | Yerleşik paneldeki kutuya tıklamanın aynısı | |
| `SetLayersVisible({...}, bool)` | Birden çok öğeyi açar/kapatır | |
| `ShowOnlyLayers({...})` | Sadece verilenler açık kalır | |
| `ResetLayersToDefaults()` | Geçerli modun varsayılanlarına döner | |
| `GetLayerState(MapLayer)` | Off / On / Mixed | `LayerState` |
| `ParentOf`, `ChildrenOf`, `TopLevelLayers`, `KindOf` | Katman ağacı (serbest fonksiyonlar) | En üst öğede `ParentOf` = nullopt |
| `LayerName`, `LayerKey`, `LayerFromKey` | Görünen ad, ayar anahtarı ("runways.water"), anahtardan öğe | |
| `IsLayerDefaultVisible(MapLayer, MapMode)` | Modun varsayılanı | `bool` |
| **Kullanıcı çizimleri** | | |
| `SetDrawTool(DrawTool)` / `GetDrawTool()` | Pan, Point, Route, Region. Değişince yarım çizim biter | |
| `AddDrawingPoint(lat, lon)` | Araca göre nokta / köşe ekler | Pan'da false |
| `FinishDrawing()` | Yarım rotayı (≥2 nokta) / bölgeyi (≥3 nokta) bitirir | Yarım çizim yoksa false |
| `UndoDrawing()` | Son köşeyi, yarım çizim yoksa son şekli siler | Silinecek yoksa false |
| `ClearDrawings()`, `HasUnfinishedDrawing()` | Hepsini siler / yarım çizim var mı | |
| `GetShapes()`, `RemoveShape(id)` | Bitmiş şekiller (id, tür, etiket P1/R1/A1, noktalar) / tek şekli siler | |
| `ExportDrawings()` / `ImportDrawings(text)` | Metin: satır başına `P\|R\|A <no> lat lon [lat lon ...]` | Import öncekileri siler |
| **Uçak, iz, rota** | | |
| `SetAircraftSource(DataSource)` / `GetAircraftSource()` | XPlane (dataref) ya da Host | |
| `SetAircraft(AircraftState)` / `GetAircraft()` | Host kaynağında uçak durumu (geçerli mi, konum, yön, yer hızı, irtifa); her karede verilir | |
| `ClearTrack()` | Uçuş izini siler | |
| `SetRouteSource(DataSource)` / `GetRouteSource()` | XPlane (FMS) ya da Host | |
| `SetRoute(waypoints, activeIndex)` | Host rotası (kimlik, konum, irtifa); activeIndex uçulan noktadır | XPlane kaynağında yok sayılır |

| Sorgu | | |
|---|---|---|
| `QueryAt(lat, lon)`, `QueryAtScreen(x, y)` | Noktadaki nesneler: katman + tooltip metni, en üstteki önce. Son çizilen kareye göre | `vector<MapObject>` |
| **Ayarlar** | | |
| `ExportSettings()` / `ImportSettings(text)` | INI metni: `[view]` mode, center_lat, center_lon, zoom, follow_aircraft; `[tools]`; `[layers.<mod>]` | Tanınmayan bölümler yok sayılır |
| `GetTerrainSettings()` / `SetTerrainSettings(TerrainSettings)` | Zemin görünümü: palet kaynağı, sınıf renkleri, gölge, geçiş, sınır kıvrımı, yakın zoom karışımı, kabartma, su, VFR rakım bantları | Bölüm 4'teki zaman kuralı |
| `ExportTerrainSettings()` / `ImportTerrainSettings(text)` | Aynı ayarlar, açıklamalı INI metni olarak | |
| `GetSettingsRevision()`, `GetDrawingsRevision()` | Ayar / çizim her değiştiğinde artan sayaç. Host kaydetme zamanını buna göre seçer | `ExportSettings` sayacı artırır |
| **Zemin ön önbelleği ve teşhis** | | |
| `PrecacheVisibleArea()`, `PrecacheArea(GeoBounds)` | Alandaki kurulu sahne karolarını diske hazırlar | Sahne yoksa / önbellek sınırının %80'ini aşarsa false |
| `CancelPrecache()` | Ön önbelleği durdurur | |
| `SetDiagnostics(bool)` / `IsDiagnosticsOn()` | Log'a periyodik teşhis satırı, durum metnine performans | |

## 7. Katman kuralları

| İşlem | Sonuç |
|---|---|
| Üst öğeyi kapatmak | Alt öğeler gizlenir; alt öğelerin kendi durumu saklanır |
| Kapalı katmanın alt katmanını açmak | Sadece o alt katman açılır; katman durumu Mixed |
| Son açık alt katmanı kapatmak | Katman da kapanır |
| Bütün alt katmanları kapalı katmanı açmak | Bütün alt katmanlar açılır |
| `SetLayerVisible(grup, true)` | Grubun bu modda varsayılan açık katmanları açılır |
| `ToggleLayer(grup)` | Varsayılan açıklardan biri kapalıysa onlar açılır; değilse gruptaki her şey kapanır |
| `ShowOnlyLayers` | Her şey kapanır; verilen katman bütün alt katmanlarıyla, verilen alt katman tek başına, verilen grup varsayılanlarıyla açılır |

| Grup | Katman (alt katmanlar) | Ayar anahtarı |
|---|---|---|
| MapGroup | Terrain (TerrainForests, TerrainBuiltUp); Roads (RoadsMotorways, RoadsPrimary, RoadsSecondary, RoadsLocal, RoadsRailways, RoadsPowerLines); Grid | terrain[.forests\|.built_up], roads[.motorways ...], grid |
| AirportsGroup | AirportSymbols; Runways (Paved, Unpaved, Water, Overruns, Markings, Numbers); Surfaces (Pavement, PavementEdges, Boundary, Markings); Objects (Lighting, Helipads, Beacons, Windsocks, Gates, TieDowns, Hangars, OtherStarts, StartNames, ApproachLights, Signs); TaxiRoutes | airports, runways[.*], airport_surfaces[.*], airport_objects[.*], taxi_routes |
| NavaidsGroup | Vor, Ndb, Ils, Fixes | vor, ndb, ils, fix |
| AirwaysGroup | AirwaysLow, AirwaysHigh | airways_low, airways_high |
| FlightGroup | Aircraft, FmsRoute, FlightTrack | aircraft, fms, track |

| | | |
|---|---|---|
| (grupsuz) | Drawings (DrawingsPoints, DrawingsRoutes, DrawingsRegions, DrawingsLabels) | drawings[.*] |

Mod başına varsayılanlar `IsLayerDefaultVisible` ile okunur.

## 8. Pencere stilleri ve giriş

| | Embedded | Standalone |
|---|---|---|
| Çerçeve | Yok | X-Plane penceresi çerçevesi |
| Taşıma / boyut | Sadece `SetGeometry` (en küçük 50 × 50) | Kullanıcı ve `SetGeometry` (en küçük 400 × 260) |
| Pop-out | Yok | `SetPoppedOut` |
| UI gizliyken | Hiçbir kontrol yok | İmleç üzerindeyken "Controls" düğmesi |

| | Fare kapalı | Fare açık |
|---|---|---|
| UI kapalı | Tıklama, sürükleme, tekerlek, sağ tık alttaki pencereye geçer | Sürükle: kaydır (4 boxel eşik); tekerlek: imleç etrafında zoom; tooltip; çizim aracında tık: nokta, sağ tık: bitir |
| UI açık | Kontroller çalışır; haritanın boş alanındaki tıklama alttaki pencereye geçer | Önce kontroller, sonra harita |

## 9. Örnek: `examples/map_api_test`

Bütün API fonksiyonlarını çağıran ayrı bir X-Plane eklenti projesi; sadece paketi kullanır.

| Dosya | Görev |
|---|---|
| `MapApiCalls.h/.cpp` | Her API çağrısı ayrı bir metot (`OpenMap`, `CloseMap`, `ZoomIn`, `SetLayerVisible` ...). Eklentinin geri kalanı haritaya sadece bu sınıfla erişir |
| `SelfTest.h/.cpp` | X-Plane içinde bütün fonksiyonları sırayla çağırır, her kontrolü Log.txt'ye `[map-api-test] PASS\|FAIL` olarak yazar |
| `TestPluginMain.cpp` | Giriş noktaları; *Plugins* > *Map API Test* menüsü aynı metotları elle çağırır (zoom, pan, mod, KSEA, katman, UI, fare, taşıma, boyut, göster/gizle) |

```
#include "PrimeMap.h"

bool MapApiCalls::OpenMap(const prime::Geometry& pGeometry, prime::WindowStyle pStyle, bool pUi,
bool pMouse)
{
    prime::OpenConfig tConfig;
    tConfig.mGeometry = pGeometry;
    tConfig.mStyle = pStyle;
    tConfig.mUiVisible = pUi;
    tConfig.mMouseInput = pMouse;
    mMap = prime::Map::Open(tConfig);
    return mMap != nullptr;
}
```

## 10. Sınırlar

| | |
|---|---|
| Harita sayısı | Çoklu: aynı anda birden çok açık `Map` penceresi |
| Çizim yüzeyi | Kendi XPLM penceresi; 3D kokpit paneline ya da başka bir pencerenin içine çizmez |

| | |
|---|---|
| Birden çok host | Kütüphane nesnesi süreçte tektir ve paylaşılır; ayrı veri yüklenmez |
| Sorgu | Tooltip metni döner; ayrı kimlik alanı yok |

## 11. Sürüm

0.22 — ekip kalıbına geçiş: SHARED DLL (`MapLibrary`, `PRIME_MAP_EXPORT`), sürüm sabitleri CMake `project(VERSION)` ile birleştirildi ve configure'da denetlenir. 0.20 — ilk sürüm (o dönem API sürümü 1.0.0 diye etiketlenmişti).